

MT3 Report

Team Name: itworks

May 4th, 2026

Summary

In Milestone 3, we implemented three required optimizations: Flash Attention, Local/Windowed Attention, and KV Caching. We also implemented five additional optimizations: Configuration Sweep, Constant Memory, `__restrict__`, Shared Memory Padding, and Host Allocation Alignment.

Analysis

req_4: Flash Attention

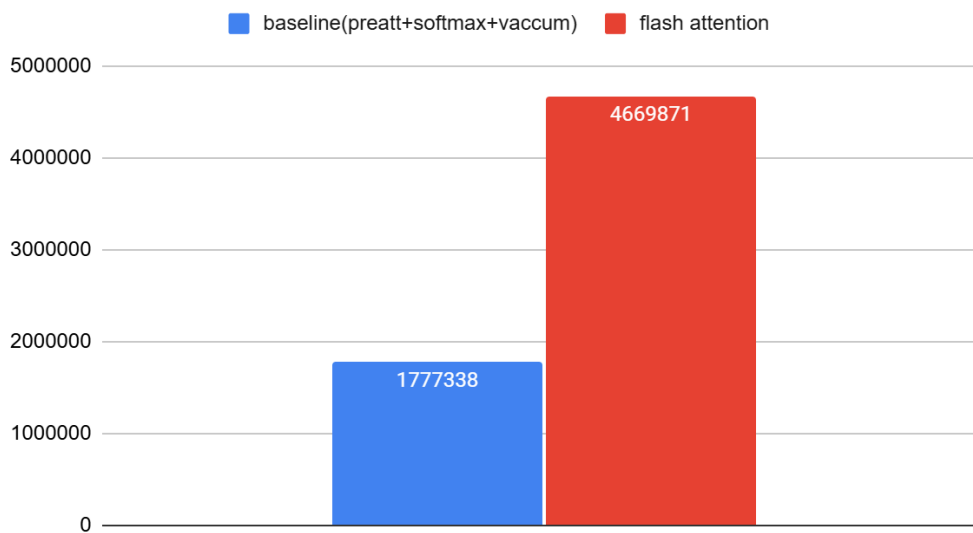
In this optimization, we implemented a FlashAttention optimization with fused kernel (based on FlashAttention-2 algorithm) to replace the original three-kernel attention pipeline. Specifically, we fused $\text{preatt} (Q \times K^T)$, softmax, and $\text{vaccum} (P \times V)$ into a single kernel, and replaced the standard softmax with an online softmax design. No other parts of the model (e.g., `matmul`, `layernorm`) were modified, so any performance difference comes solely from the attention implementation. We check the correctness of our flash attention design by running `test_gpt2` program with optimized attention, and the error is under a certain threshold.

From profiling results, the baseline attention takes approximately 1.78 ms in total (1.214 ms for `preatt`, 0.369 ms for softmax, and 0.195 ms for `vaccum`), while the fused FlashAttention kernel takes 4.67 ms. This means the optimized version is about $2.6\times$ slower than the baseline for the attention component.. Since all other kernels remain unchanged, we will only analyze the change to attention kernel itself.

The main reason for this result is that the sequence length in our model is relatively small ($T = 64$), so the attention matrix (64×64) is too small for memory traffic to become a bottleneck. Since FlashAttention do not aim to improve time complexity but improve the memory usage, In the FlashAttention paper, the authors evaluate their method on much larger sequence lengths (e.g., $T \geq 512$), and the benefit of flash attention increases as T increases. So that when $T = 64$, the improvement that is not significantly enough is an expected behavior.

Moreover, the FlashAttention kernel introduces additional overhead from online softmax computation. In our implementation, only a subset of threads in each block are responsible for updating the softmax values (e.g., row-wise max and normalization factor), which introduces control divergence and increases per-thread workload. Furthermore, to avoid race conditions, parts of the computation such as softmax updates and PV accumulation are serialized within a block. This design significantly reduces parallelism compared to the baseline implementation, where matrix multiplications and softmax are highly parallelized across threads. As a result, the fused kernel sacrifices parallelism for correctness and fusion, and the overhead from reduced parallelism, synchronization, and increased per-thread computation outweighs the benefit of reduced global memory access in this workload.

Flash attention(ms) vs baseline attention(ms)



req_5: local/windowed Attention

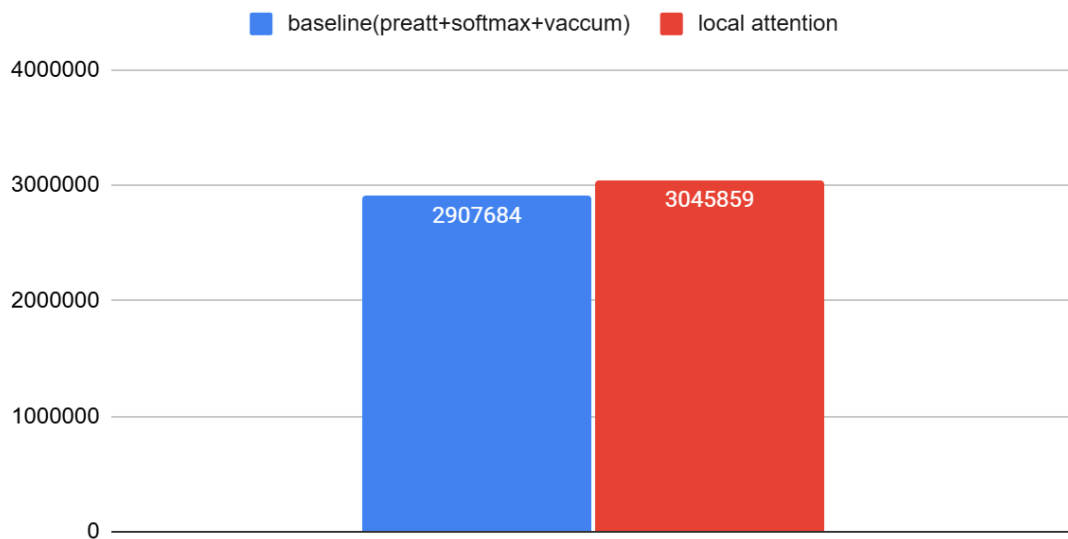
In this optimization, we implemented local (windowed) attention by restricting each query token to attend only to a fixed-size ($W = 128$) causal window of previous tokens, instead of the full sequence. We fused the original attention pipeline (preatt, softmax and vaccum) into a single kernel. In our design, each (b, nh, q_t) query position is assigned to one CUDA block, and threads within the block are responsible for different positions in the local window. This allows the computation over the window dimension to be parallelized within the block. In this fused kernel, several threads will contribute to the same output, we use atomic add to avoid race condition.

From the profiling results, Among all test cases, the optimized local attention kernel takes approximately 3.05 ms, while the baseline attention pipeline (preatt, softmax and vaccum) takes about 2.91 ms in total. This shows that the optimized local attention implementation is close to

the baseline performance but still slightly slower (about $1.05\times$ slower). Since no other parts of the model are modified, this difference can be attributed directly to the local attention kernel.

The main reason local attention does not outperform the baseline is that its theoretical advantage is not fully realized in our workload. The sequence length is relatively small and comparable to the window size (like the test contains case $T = 35$, where windowed reduced to standard attention), so the actual reduction in computation is limited. At the same time, the fused local attention kernel introduces additional overhead, including atomic operations when accumulating output values, syncthread within each block, and shared-memory reductions for softmax. However, our implementation still shows a complexity improvement from $O(T^2)$ to $O(TW)$. It will be better shown if we test it with longer sequence length (when $T \gg W$).

local attention(ms) vs baseline(ms) among all local_attn_verify testcases



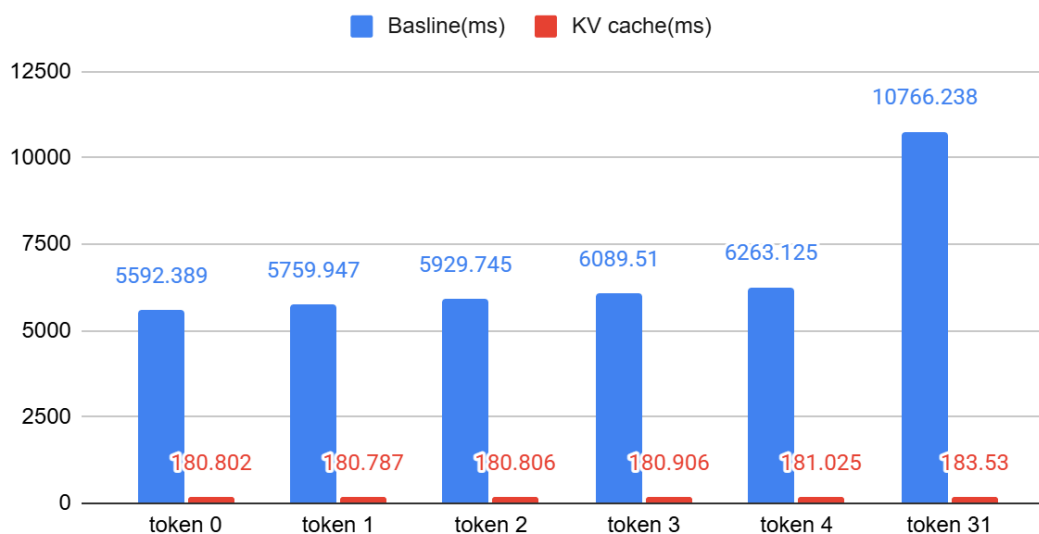
req_6: KV Cache Optimization

In this optimization, we implemented a KV cache optimization to accelerate forwarding. Our implementation has two phases: prefill and decode. During the prefill phase, the model processes the initial input sequence and computes all attention keys and values. These keys and values are then stored in a pre-allocated cache (we use a maximum length of 1024 tokens). During the decode phase, instead of recomputing keys and values for all previous tokens, the model reuses the cached KV tensors and only computes the key and value for the newly generated token. This significantly reduces redundant computation across decoding steps.

The correctness of the KV cache implementation is verified through both deterministic tests and randomized tests. We compared the outputs of the cached version against the baseline implementation without KV caching, and confirmed that they match within a certain threshold. This is expected because, under a causal attention mask, the keys and values corresponding to previous tokens remain unchanged once computed. Therefore, reusing cached KV tensors does not alter the model's functional behavior, but only avoids recomputation.

From a performance perspective, KV caching changes the computational complexity. In the baseline implementation, each decoding step recomputes attention over the entire sequence, resulting in a total complexity of $O(T^2)$ across T decoding steps. With KV caching, each step only processes the new token against the cached sequence, reducing the total complexity to $O(T)$ attention computations. This theoretical improvement is also reflected in practice, where the decode phase becomes significantly faster as sequence length increases and only grows linearly. This optimization demonstrates that avoiding redundant computation across time steps is critical for efficient transformer inference, especially in long-sequence generation scenarios.

Baseline(ms) vs KV cache(ms) in case random B = 32, Prompt =32, Gen = 32



op_7: Configuration Sweep / Optimization

In op_7, we implemented the configuration sweep by tuning the compile tile parameters of the matmul kernel and find a better combination with better performance. In req_0, the matmul kernel configuration is:

```
#define U 16
```

```
#define BLOCK_SIZE 256
#define TILE_WIDTH 16
```

We want to find new parameter choices for balance between memory use and arithmetic intensity and register pressure. We wrote a python sweep script to evaluate different combination of U and TILE_WIDTH. The script also compute the shared-memory usage as:

```
smem = (BLOCK_SIZE * TILE_WIDTH + TILE_WIDTH * U) * sizeof(float)
```

Make sure the configuration is valid. It also generate a new matmul with a template, running test_gpt2_kernels and record the result into a csv file.

The script also backs up the original before the sweep and restores it afterward. We swept the following parameters:

```
U ∈ {8, 16, 32}
TILE_WIDTH ∈ {8, 16, 32}
```

Configuration sweep results for op_7

U	TILE_WIDTH	BLOCK-SIZE	Static Shared Memory	Runtime
8	8	64	2.2KB	1.5806 s
8	16	128	8.5KB	1.0367 s
8	32	256	33.0KB	1.0691 s
16	8	128	4.5KB	1.0374 s
16	16	256	17.0KB	1.0238 s
32	8	256	9.0KB	1.0205 s
32	16	512	34.0KB	1.0331 s

The new configuration with less runtime and static shared memory is:

```
#define U 32
#define BLOCK_SIZE 256
#define TILE_WIDTH 8
```

The rest of the kernel is unchanged. So the correctness does not change based on the kernel and end to end theorem. This optimization does not change the result but doubles the number of output channels computed per thread while reducing the K-tile depth from 16-8.

Nsight Compute profiling summary for Configuration sweep

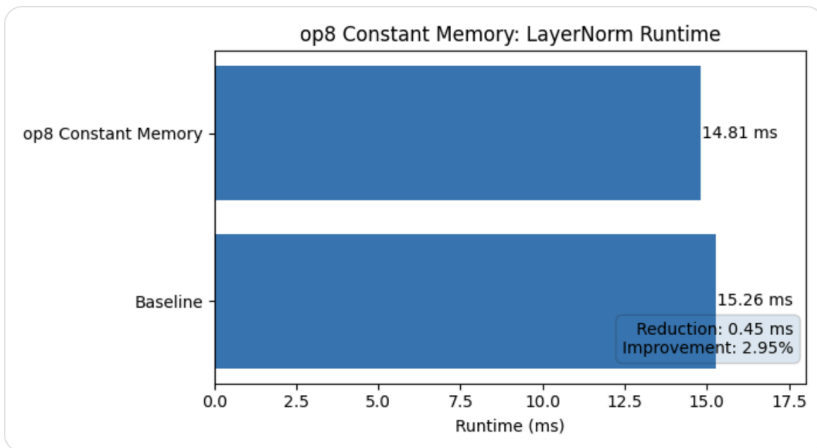
Metric	req_0	op_7
--------	-------	------

Duration, OC=768	953 μ s	605 μ s
Memory Throughput	69.85%	57.22%
Compute SM Throughput	32.79%	47.34%
FFMA inst/cycle	365.49	574.15
Warp Cycles / Issued Inst	29.28	13.82
Stall MIO Throttle	16.29	5.44
Stall Long Scoreboard	3.61	4.23
Registers / Thread	40	56
Shared Memory / Block	17.41 KB	9.22 KB
Active Warps / Scheduler	2.93	2

According to the profiling data from op_12 and req_0, this sweep reduced the dominant bottleneck in matmul invocation, Stall MIO Throttle dropped from 16.29 to 5.44 cycles per instruction. Across all 49 matmul calls, the total matmul time decreased from 56.27 ms to 47.96 ms, corresponding to a 14.8% reduction.

op_8: constant memory

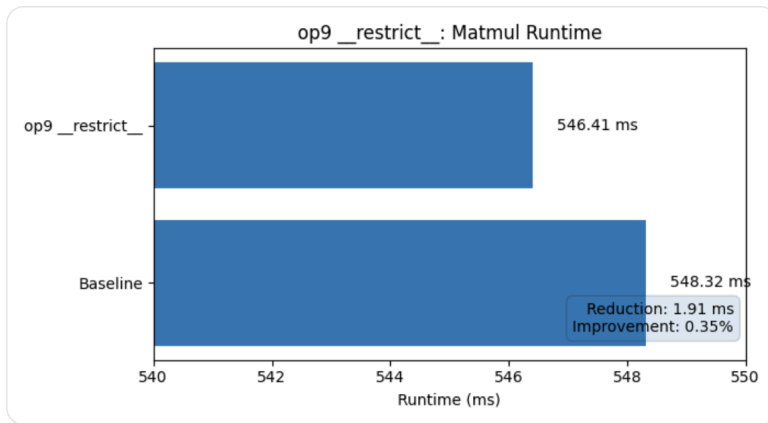
For op_8, we optimized LayerNorm by moving its weight and bias arrays into CUDA constant memory. In the original implementation, every LayerNorm call reads the weight and bias values from global memory. These arrays are small, read-only, and repeatedly accessed by many threads, so they are a good fit for CUDA constant memory. We added `__constant__` arrays for the LayerNorm weight and bias and used `cudaMemcpyToSymbol` before launching the LayerNorm kernel so that the kernel could read these values from constant memory instead of regular global memory. This does not change the mathematical computation, since the same weight and bias values are used. In profiling, LayerNorm time decreased from about 15.26 ms to 14.81 ms, which is about a 3% improvement for the LayerNorm kernel. However, the end-to-end improvement is limited because LayerNorm only accounts for a small percentage of total runtime, while matmul still dominates.



op_9: restrict

For op_9, we added `__restrict__` qualifiers to CUDA kernel pointer arguments where the input and output tensors are known not to overlap. Without `__restrict__`, the compiler has to conservatively assume that two pointers might alias, meaning that a write through one pointer could affect data read through another pointer. This can prevent the compiler from safely reusing loaded values or reordering memory operations. By marking non-overlapping pointers as `__restrict__`, the compiler gets stronger aliasing information and can optimize memory accesses more aggressively. This optimization also does not change the mathematical result, since it only provides additional information to the compiler. We add `ptx` flag to `nvcc` compiler and generated an intermediate representation (assembly-like code). We found that the load instruction changes (from `ld.global.u32` to `ld.global.nc.u32`), this is a compiler & ISA-level optimization and we can't explicitly analyze its efficiency.

In profiling, `matmul` time decreased slightly from about 548.32 ms to 546.41 ms, and `LayerNorm` time decreased from about 15.26 ms to 14.72 ms. The improvement is modest because the main bottleneck is still memory bandwidth rather than pointer aliasing overhead.



op_12: Shared Memory Padding

In op_12, we optimized the req_0 matmul kernel by adding one padding element to the inner dimension of the shared-memory input tile, the req_0 configuration is:

```
#define U 16
#define BLOCK_SIZE 256
#define TILE_WIDTH 16
```

We add one padding:

```
input_s[BLOCK_SIZE][TILE_WIDTH] → input_s[BLOCK_SIZE][TILE_WIDTH + 1]
```

The rest of the kernel is unchanged. The thread mapping, global-memory indexing and logic are identical to the req_0 implementation. So this optimization does not affect correctness. It only changes the physical layout of shared memory.

This fixes a bank conflict in the access pattern `input_s[threadIdx.x][k]`. Without padding, the row stride is 16, so the bank index is:

```
bank = (threadIdx.x * 16 + k) % 32
```

This maps one warp onto only two shared-memory banks, creating severe serialization. Each of those two banks is accessed by 16 threads which means a 16 way bank conflicts in req_0. With padding, the stride becomes 17:

```
bank = (threadIdx.x * 17 + k) % 32
```

Since $\text{gcd}(17, 32) = 1$, the 32 warp threads spread across all 32 banks, eliminating the conflict.

Nsight Profiling summary for shared-memory padding

Metric	req_0	op_12
Shared-memory bank conflicts	26,542,080	0
Static shared memory / block	17.41 KB	18.43 KB

Compute SM Throughput	32.79%	40.38%
FFMA inst/cycle	365.49	450.04
Kernel duration, OC=768	953 μ s	770 μ s
Total matmul time, 49 calls	56.27 ms	46.01 ms

Nsight Compute confirms this computation: shared-memory bank conflicts dropped from 26,542,080 in req_0 to 0 in op_12. The performance improvement is visible across different shapes in all major matmul layers: attention projection, QKV projection, FFN up projection and Vocab projection with a 1.2x to 1.24 speed up. Total matmul time decreased from 56.27 ms to 46.01 ms, an 18.2% improvement. The Compute SM Throughput increases from 32.79% to 40.38% showing that after padding, the SM spent less time stalled on shared-memory accesses and more time issuing useful arithmetic instructions. The cost of padding is small:

$$256 \text{ rows} * 1 \text{ extra float} * 4 \text{ bytes} = 1024 \text{ bytes}$$

The cost is only 1 KB extra shared memory per block, which is also verified in the profiling data, making this a low-cost, high-impact optimization.

op_13: host allocation alignment

For op_13, we modified the host-side tensor allocation logic to align each parameter and activation tensor to a 128-byte boundary. The original allocation code placed many tensors inside one large contiguous allocation, but the start of each tensor was determined by the previous tensor's size. If a tensor starts at a misaligned address, even coalesced memory accesses can require extra memory transactions or extra L2 sectors. To reduce this issue, we added an align_up helper and padded each tensor's offset to 32 floats, which equals 128 bytes. This keeps tensor shapes and indexing unchanged while improving the alignment of each tensor's starting address. In profiling, matmul time decreased slightly from about 548.32 ms to 545.76 ms, while LayerNorm stayed almost the same. This suggests that alignment helped reduce some memory inefficiency, but it did not remove the main memory-bound behavior of the matmul kernel.

